COMP2040: Operating Systems Homework 01: Process Synchronization and Deadlocks

Daniela Mendez Ramirez
College of Engineering and Computer Science
VinUniversity, Hanoi, Vietnam

***Abstract*—This homework studies process synchronization and deadlocks. Part A examines race conditions, critical-section requirements, semaphore behavior in a bounded buffer, and condition variables. Part B applies the Banker's safety and resource-request algorithms, the deadlock-detection algorithm, and resource-allocation graphs. The optional programming task implements a bounded buffer in C using a mutex and two condition variables. The report shows the reasoning and intermediate states used to reach each answer.**

***Index Terms*—Operating Systems, Process Synchronization, Race Conditions, Semaphores, Condition Variables, Banker's Algorithm, Deadlocks, POSIX Threads.**

I. Introduction

Concurrent threads may access shared data at overlapping times. Without correct synchronization, their operations can interfere and produce incorrect results. Operating systems use synchronization tools such as mutex locks, semaphores, and condition variables to coordinate access to shared resources.

Deadlocks occur when a group of processes cannot continue because each process is waiting for a resource held by another process. This homework studies these problems through execution interleavings, resource-allocation states, and algorithms.

The report follows the order of the assignment. Intermediate calculations are included because the final result alone does not show how each conclusion was obtained.

II. Part A: Synchronization

A. Question 1: Race Condition

1) (a) Interleaving in which both threads book the seat: Initially, `seats = 1`. The condition check and the later read, update, and write are separate operations. The following interleaving allows both threads to pass the condition:

Table I
Interleaving that causes two successful messages

Step	Thread A	Thread B	seats
1	checks `seats > 0` (true)		1
2		checks `seats > 0` (true)	1
3	reads `seats` into `r`		1
4		reads `seats` into `r`	1
5	sets its `r = 0`		1
6		sets its `r = 0`	1
7	writes `seats = 0`		0
8		writes `seats = 0`	0

Both threads print `booked`, even though only one seat existed. In this interleaving, the final value is `seats = 0`. It cannot be 1 or -1 when both threads execute the shown successful path: each thread reads the original value 1 and computes 0, so each write stores 0. The error is that the program reports two bookings for one seat, not that the final counter becomes negative.

2) (b) Correct placement of the lock: The lock must protect the condition check and the complete read-update-write operation. It must be acquired before testing `seats` and released only after the decision and update are complete:

```
lock(m);
if (seats > 0) {
    r = seats;
    r = r - 1;
    seats = r;
    printf("booked\n");
}
unlock(m);
```

The condition check must be inside the critical section. If only the read, update, and write lines are protected, both threads may evaluate `seats > 0` before either obtains the lock. The second thread can then enter after the first finishes, even though its earlier condition check is no longer valid. Protecting the check as well forces the second thread to test the updated value.

B. Question 2: Critical-Section Requirements

The three requirements considered here are mutual exclusion (at most one thread is in the critical section), progress (a thread outside the critical section cannot prevent an eligible thread from entering), and bounded waiting (a thread cannot be postponed forever after requesting entry).

1) (a) Protocol 1: Protocol 1 uses strict alternation. Mutual exclusion is preserved, but **progress can fail**. For example, let `turn = 0`. T0 enters and leaves its critical section, setting `turn = 1`. If T1 does not want to enter, T0 may request entry again but must wait while `turn != 0`. T1 is not interested in the critical section, yet its turn prevents T0 from entering. This violates progress.

2) (b) Protocol 2: Protocol 2 does not guarantee **mutual exclusion**. The read of `busy` and the later assignment `busy = true` are not one atomic operation. An execution can be:

1) T0 reads `busy` as false.

2) T1 reads `busy` as false.
3) T0 sets `busy = true` and enters.
4) T1 sets `busy = true` and also enters.

Both threads can therefore be in the critical section at the same time.

3) (c) Protocol 3: Protocol 3 guarantees **mutual exclusion**, because a thread enters only after observing the other thread's flag as false. However, **progress can fail**. If T0 sets `flag[0] = true` and T1 sets `flag[1] = true` before either checks the other's flag, both see a true flag and both wait forever. Neither thread is in the critical section, but neither can proceed.

4) (d) Why Peterson's solution avoids this failure: Peterson's solution adds `turn = j` after setting the thread's own flag. If both threads want to enter, the last assignment to `turn` gives priority to the other thread. Thus, both threads cannot remain waiting on each other's flag at the same time: one is allowed to enter, and the other waits until the first leaves.

C. Question 3: Semaphores and the Bounded Buffer

The buffer has three slots. Initially, `empty = 3`, `full = 0`, and `mutex = 1`. The buffer is shown with the oldest item first. When a signal hands a permit to a waiting thread, the semaphore value does not increase; the permit is transferred to that thread.

1) (a) Completed state table: At step 2, the producer inserts item 1 and signals `full`, waking C1. C1 then completes its removal at step 3. At step 7, the producer cannot obtain an empty-slot permit and blocks. At step 8, C2 removes the oldest item, which releases an empty-slot permit and wakes P. P uses that permit to insert item 5.

2) (b) Blocking and wake-up events:

- At step 1, C1 blocks on `full`, because the buffer is empty. P wakes C1 at step 2 by signaling `full`.
- At step 7, P blocks on `empty`, because all three slots are occupied. C2 wakes P at step 8 by signaling `empty` after removing an item.

3) (c) Reversing the consumer semaphore order: If a consumer calls `wait(mutex)` before `wait(full)`, C1 can acquire the mutex while the buffer is empty and then block on `full` without releasing the mutex. The producer can obtain an empty-slot permit but cannot acquire the mutex to insert an item. C1 is waiting for an item that the producer cannot insert, while the producer is waiting for the mutex held by C1. This causes deadlock.

D. Question 4: Condition Variables

1) (a) Execution that exposes the bug: The buffer starts empty. The code uses `if`, so a consumer checks the condition only once.

1) Consumer C1 locks the mutex, sees `count == 0`, and waits on `not_empty`. The wait releases the mutex.
2) Consumer C2 also locks the mutex, sees `count == 0`, and waits on `not_empty`.
3) The producer locks the mutex, inserts one item, increments `count` to 1, signals `not_empty`, and unlocks the mutex. Assume C1 is awakened.
4) C1 reacquires the mutex, removes the only item, and decreases `count` to 0.
5) C2 may now wake and reacquire the mutex. Because the code uses `if`, it does not check `count` again and calls `remove_item()` while the buffer is empty.

The bug is that waking up does not guarantee that an item is still available when the consumer reacquires the mutex.

2) (b) Corrected code and spurious wake-ups: The condition must be checked again after every wake-up. The corrected function is:

```
int get(void) {
    int item;

    lock(m);
    while (count == 0) {
        wait(not_empty, m);
    }
    item = remove_item();
    unlock(m);
    return item;
}
```

The `while` loop checks the condition whenever the thread returns from the wait. If another consumer has already removed the item, or if the thread wakes without a signal (a spurious wake-up), `count` is still zero and the consumer waits again. The item is removed only after the condition is true while the mutex is held.

III. Part B: Deadlocks

A. Question 5: Banker's Algorithm

The resource totals are $(A, B, C) = (11, 13, 5)$. The initial Available vector is $(2, 4, 1)$. The Need matrix is calculated as:

$$Need = Max - Allocation.$$

Table III
NEED MATRIX

Process	A	B	C
P0	5	1	1
P1	3	1	2
P2	3	4	2
P3	3	4	1
P4	1	0	0

1) (a) Need matrix:

2) (b) Safety algorithm: The algorithm starts with `Work = Available = (2,4,1)`. A process can finish when its Need is less than or equal to Work in every resource type. When it finishes, its Allocation is added to Work.

Table IV
SAFETY ALGORITHM STEPS

Step	Process	Need	Work before	Work after
1	P4	(1,0,0)	(2,4,1)	(4,4,1)
2	P3	(3,4,1)	(4,4,1)	(7,7,1)
3	P0	(5,1,1)	(7,7,1)	(10,9,4)
4	P1	(3,1,2)	(10,9,4)	(10,12,5)
5	P2	(3,4,2)	(10,12,5)	(11,13,5)

Table II
SEMAPHORE AND BUFFER STATE AFTER EACH EVENT

Step	Event	empty	full	Buffer	Waiting
0	Initial state	3	0	[]	–
1	C1 calls remove; it blocks on `full`	3	0	[]	C1 on full
2	P inserts item 1; signal wakes C1 (permit handed off)	2	0	[1]	–
3	C1 finishes removing item 1; signals `empty`	3	0	[]	–
4	P inserts item 2	2	1	[2]	–
5	P inserts item 3	1	2	[2, 3]	–
6	P inserts item 4	0	3	[2, 3, 4]	–
7	P calls insert(item 5); it blocks on `empty`	0	3	[2, 3, 4]	P on empty
8	C2 removes item 2; P receives the empty permit and inserts item 5	0	3	[3, 4, 5]	–

Every process can finish, so the state is safe. One safe sequence is:

$$\boxed{P4 \to P3 \to P0 \to P1 \to P2}$$

3) (c) P3 requests (0,1,1): The request is $(0, 1, 1)$.

First bound:

$$(0, 1, 1) \leq Need_{P3} = (3, 4, 1)$$

The request does not exceed P3's remaining declared maximum.

Second bound:

$$(0, 1, 1) \leq Available = (2, 4, 1)$$

The resources are currently available, so the request passes both bounds.

After the tentative allocation:

$$Available' = (2, 4, 1) - (0, 1, 1) = (2, 3, 0)$$

$$Allocation'_{P3} = (3, 3, 0) + (0, 1, 1) = (3, 4, 1)$$

$$Need'_{P3} = (3, 4, 1) - (0, 1, 1) = (3, 3, 0)$$

The other processes remain unchanged. The safety test can finish P4 first, then P3, followed by P0, P1, and P2:

$$(2, 3, 0) \xrightarrow{P4} (4, 3, 0) \xrightarrow{P3} (7, 7, 1) \xrightarrow{P0} (10, 9, 4) \xrightarrow{P1} (10, 12, 5) \xrightarrow{P2} (11, 13, 5)$$

The tentative state is safe. Therefore, **P3's request is granted**.

4) (d) P0 requests (0,1,1) from the original state: The request is within both bounds:

$$(0, 1, 1) \leq Need_{P0} = (5, 1, 1)$$

$$(0, 1, 1) \leq Available = (2, 4, 1)$$

The tentative state would have:

$$Available' = (2, 3, 0)$$

$$Allocation'_{P0} = (3, 3, 4)$$

$$Need'_{P0} = (5, 0, 0)$$

The safety test starts with `Work = (2,3,0)`. P4 can finish because its Need is $(1, 0, 0)$. It releases $(2, 0, 0)$, giving:

$$Work = (2, 3, 0) + (2, 0, 0) = (4, 3, 0)$$

No remaining process can finish:

- P0 needs $(5, 0, 0)$, but only 4 units of A are available.
- P1 needs $(3, 1, 2)$, but no C is available.
- P2 needs $(3, 4, 2)$, but B and C are insufficient.
- P3 needs $(3, 4, 1)$, but B and C are insufficient.

The state is unsafe, so **P0's request must wait**. The same request is safe for P3 because P3's new Need is small enough to allow it to finish after P4. For P0, the tentative allocation leaves P0 needing five units of A, while only four can become available after P4 finishes. Thus, the process order and the resulting remaining Need matter, not only the request vector itself.

B. Question 6: Deadlock Detection and Recovery

The initial Available vector is:

$$Work = (0, 1, 0).$$

For the detection algorithm, a process can finish if its outstanding Request is less than or equal to Work. Its Allocation is then released and added to Work.

1) (a) Detection algorithm: P0 has Request $(0, 0, 0)$, so it can finish and release $(0, 1, 2)$. P1 can then finish because its Request $(0, 2, 1)$ is less than or equal to the new Work $(0, 2, 2)$. It releases $(0, 1, 1)$.

Table V
DEADLOCK DETECTION STEPS

Step	Process	Request	Work before	Work after
0	Initial	–	–	(0,1,0)
1	P0	(0,0,0)	(0,1,0)	(0,2,2)
2	P1	(0,2,1)	(0,2,2)	(0,3,3)

The remaining requests cannot be satisfied:

$$Request_{P2} = (2, 1, 1), \quad Request_{P3} = (1, 0, 2), \quad Request_{P4} = (2, 2, 0)$$

All three require at least one unit of A, but `Work` has zero units of A. Therefore, the system is deadlocked, and the deadlocked set is:

$$\boxed{\{P2, P3, P4\}}$$

2) (b) Aborting exactly one process: Aborting a process releases its current Allocation. The available vector after each possible abort is:

- Abort P2: $(0, 3, 2)$. P3 and P4 still need A, but no A is available. The deadlock remains.
- Abort P3: $(1, 2, 2)$. P2 and P4 each need two units of A, but only one is available. The deadlock remains.
- Abort P4: $(1, 3, 0)$. P2 needs C and P3 needs C, but no C is available. The deadlock remains.

Therefore, **aborting any one of the three deadlocked processes alone does not end the deadlock**. No single candidate is sufficient. At least two of the deadlocked processes must be removed (or another recovery action must release additional resources). This follows directly from the remaining requests and the resources released by each individual process.

C. Question 7: Resource-Allocation Graphs

1) (a) Two resource types and three threads: R1 has one instance and R2 has two instances. Initially:

- T1 holds one instance of R2 and requests R1.
- T2 holds R1 and requests an instance of R2.
- T3 holds the other instance of R2 and requests nothing.

The graph contains the cycle:

$$T1 \rightarrow R1 \rightarrow T2 \rightarrow R2 \rightarrow T1.$$

However, the system is **not deadlocked**. R2 has two instances, and T3 is not waiting for another resource. T3 can finish and release its instance of R2. T2 can then obtain R2, finish, and release R1 for T1. With multiple resource instances, a cycle alone is not sufficient to prove deadlock.

If T3 also requests R1, then T3 cannot finish because R1 is held by T2. T2 waits for an instance of R2, while both instances of R2 are held by T1 and T3. T1 and T3 both wait for R1. No thread can proceed, so the system is deadlocked. The cycle now represents a deadlock because every relevant resource instance is held by a thread that is also waiting.

2) (b) Original deadlock example: shared laboratory equipment: Consider two laboratory jobs that use two pieces of equipment: a sample-preparation station (S) and an analyzer (A). Each machine can be used by only one job at a time.

Job J1 reserves S and then requests A. Job J2 reserves A and then requests S. Neither job can continue, because each is waiting for the machine held by the other.

The four necessary conditions are present:

1) **Mutual exclusion:** Each machine can be reserved by only one job at a time.
2) **Hold and wait:** J1 holds S while waiting for A, and J2 holds A while waiting for S.
3) **No preemption:** A machine is not taken away from a job automatically; the job must release it.
4) **Circular wait:** J1 waits for A held by J2, while J2 waits for S held by J1.

One way to prevent this deadlock is to break hold and wait. A job must reserve both machines before it starts using either one. If both are not available, it reserves neither and tries again later. This prevents a job from holding one machine while waiting for the other.

IV. Bonus: Bounded Buffer as a Monitor

A. Implementation

The provided C program was completed using one mutex and two condition variables. The shared data is grouped in a `BoundedBuffer` structure rather than stored in global variables. Producer and consumer threads receive pointers to this structure.

The `put()` function locks the mutex and waits in a `while` loop while the buffer is full. It inserts the item, updates the circular index and item count, signals `not_empty`, and unlocks the mutex.

The `get()` function follows the corresponding consumer logic. It waits while the buffer is empty, removes the oldest item, updates the circular index and item count, signals `not_full`, and unlocks the mutex.

B. Compilation and execution

The required build command is:

```
gcc -std=c11 -Wall -Wextra -pthread hw1_buffer.c -o
    hw1_buffer
./hw1_buffer
```

The expected successful output has equal produced and consumed sums, zero remaining items, and the final line `OK`. The exact terminal output from the local run should be inserted below after compiling and executing the submitted source file.

```
Produced sum: 400020000
Consumed sum: 400020000
Items left in buffer: 0
OK
```

C. Why both waits use while loops

A condition-variable wait may return after a spurious wake-up. Also, another thread may change the buffer before the awakened thread reacquires the mutex. Therefore, returning from `pthread_cond_wait()` does not prove that the condition is true. The `while` loop checks the condition again while holding the mutex. A producer inserts only when there is space, and a consumer removes only when an item exists.

V. Conclusion

The synchronization questions show why shared state must be protected as one complete operation and why a wake-up does not automatically mean that a condition is still true. Semaphores coordinate access to available and occupied buffer slots, while condition variables allow threads to wait for a state change under a mutex.

The deadlock questions show that safety depends on the resources a process may still need, not only on the resources

currently available. The Banker's algorithm checks whether all processes can finish in some order. The detection algorithm identifies processes that cannot proceed from the current resource state. Resource-allocation graphs also require care when resource types have multiple instances: a cycle is not always sufficient to prove deadlock.

The bonus implementation applies these concepts in C by protecting the bounded buffer with a mutex and using condition-variable loops to handle full and empty states safely.

AI Tool Disclosure

An AI assistant was used to help interpret the assignment requirements, organize explanations, review the synchronization and deadlock calculations, and improve the clarity of the report and C implementation. The submitted source code and mathematical reasoning should be reviewed and tested by the author before submission.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] The Open Group, "pthread_cond_wait," *The Open Group Base Specifications*. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_cond_wait.html

[3] The Open Group, "pthread_mutex_lock," *The Open Group Base Specifications*. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_mutex_lock.html

[4] Free Software Foundation, "GCC, the GNU Compiler Collection." [Online]. Available: https://gcc.gnu.org/